KYC / eKYC — DigiLocker (Surepass) & AEPS eKYC

InstantMudra Enterprises Admin Portal

DigiLocker = Surepass — one integration, two names in the top-level docs

DigilockerController (/api/digilocker) → DigilockerServiceImpl → https://kyc-api.surepass.app/api/v1/digilocker/*

Both call sites use one static bearer token (@Value("${surepass.digilocker.token}")) — "DigiLocker" and "Surepass" in the top-level review are the same vendor, not two.

⚠ DigilockerServiceImpl — static token, zero response handling

Static, non-refreshing bearer token

digilockerToken injected once from application.properties (a long-lived Surepass JWT — documented, masked, in Module 1). No refresh/regeneration logic anywhere in this class — when the token expires, every DigiLocker call fails until someone manually rotates the config value.

No response/error handling

`return response.getBody();` — both initializeDigilocker() and downloadAadhaar() return the raw body directly: no HTTP status check, no null check, no try/catch. A vendor error surfaces as an unhandled exception or a silent null downstream.

⚠ CRITICAL — authenticated IDOR exposing Aadhaar data

`GET /api/digilocker/download-aadhaar/{clientId}`

Requires a valid AESFilter-passed token — this path is absent from both whitelists (Module 4), so it's not anonymously public.

But nothing in DigilockerController or DigilockerServiceImpl checks that the requested clientId belongs to the caller's own KYC session or hierarchy — clientId is taken as a raw path variable and forwarded straight to Surepass.

Any authenticated user — any hierarchy level — can retrieve any OTHER person's Aadhaar data (name, photo, DOB, address) by supplying a different clientId. This is more consequential than a wallet-balance IDOR: Aadhaar is government-ID PII.

AepsEkycOrchestrator — correctly wired, injected, and never called

AepsEkycController autowires **both** aepsEkycOrchestrator and aepsEkycService — but all 5 endpoints (send-otp, validate-otp, resend-otp, biometric, validate-otp-biometric) call **only** aepsEkycService.XXX(req).

Confirmed by search: "AepsEkycOrchestrator" appears in exactly 2 files — its own definition, and this unused @Autowired field.

AepsEkycOrchestrator itself is correctly built: resolves VendorRouting → VendorCredentials → AepsVendorFactory → calls the resolved vendor. Unlike Module 5's AEPS-transaction bypass (a separate legacy class did the work), here the dead orchestrator was wired into the dependency graph and simply never invoked — someone injected it and never wrote the call.

AepsEkycServiceImpl duplicates the orchestrator's vendor-routing logic independently

The class that's actually called resolves VendorRouting itself — vendorRoutingRepository.findBySubProductId(...) appears at 5 separate call sites within AepsEkycServiceImpl (624 lines), each duplicating what AepsEkycOrchestrator already does once.

Same "pattern exists, real traffic bypasses it" shape documented for AEPS transactions in Module 5 — now confirmed for eKYC too.

☐ Standard / correctly wired ☐ Flagged risk / dead code / duplication ☐ Critical — confirmed PII-exposure gap